



算法设计与分析

讲授内容：

绪论——从“程序实现”到“算法设计”

教师：卜晨阳

联系方式：chenyangbu@hfut.edu.cn

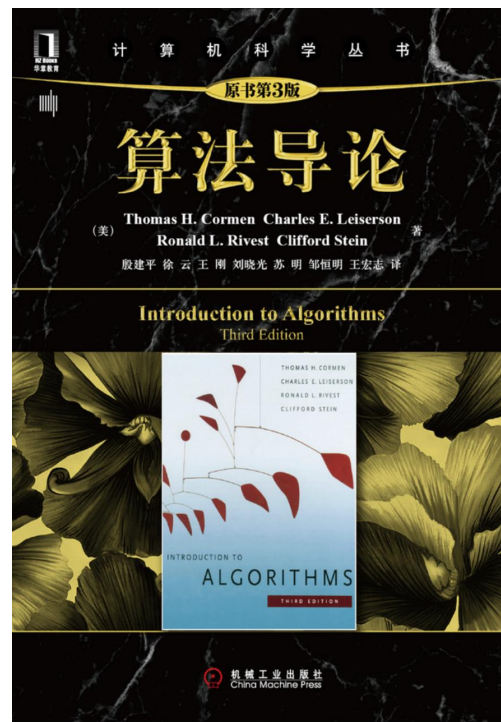
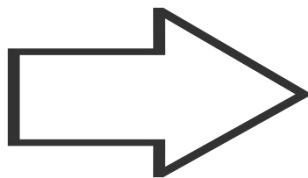
2025年11月5日

优秀的程序员 $\neq$ 高效的算法设计师

- 优秀的程序员能实现复杂功能。
- 但这门课关乎的是在写代码之前的思考。
- 目标：从“实现者”(Coder)到“设计师”(Designer)的思维跃迁

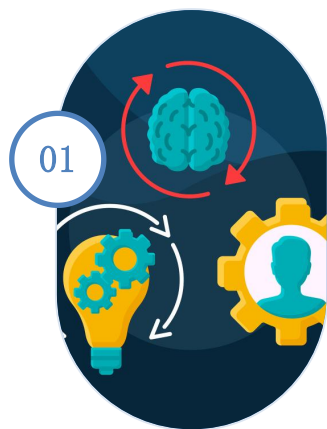


Coder



Designer

常见思维误区



代码先行，思维滞后

在项目开发中，开发者可能会急于编写代码而忽视了深入思考问题本质，导致后续需要频繁修改代码，造成资源浪费和效率低下。



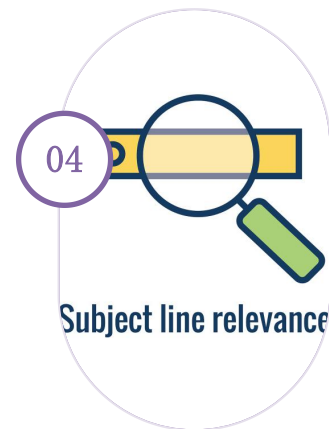
缺乏抽象，无法形式化

当开发者未能将问题抽象化，无法用形式化的方法来描述和解决问题时，可能会导致解决方案缺乏通用性和可扩展性。



不善于搜索，重复造轮子

缺乏有效的信息检索能力，开发者可能会重复发明已有的解决方案，从而浪费时间和精力，而不是站在巨人的肩膀上前进。



证明缺失，盲目自信

在缺乏充分证据或逻辑证明的情况下，盲目自信自己的想法或解决方案是正确的，可能会导致错误的决策和项目失败。

误区一：代码先行，思维滞后

- 现象： 打开IDE就开始写代码，调试代替思考。
- 根源： 混淆“实现”与“设计”。
 - 不经设计的编码是最高成本的调试：过早陷入编码细节会让你迷失在语法和边界条件中，浪费大量时间修补一个原本就脆弱的设计。设计阶段的思考成本远低于编码后的调试成本。
- 案例： 线性时间选择——“找数组中第K大的数”
 - 错误路径 (Code-First): 开始编码 -> 冒泡排序 -> 取第K个 -> 效率低下？
 - 打开IDE，写一个双重循环的冒泡排序。 -> $O(n^2)$
 - 排序后取第`arr[length - k]`个元素。
 - 发现效率极低，且如果数组很大，效率较低。
 - 推倒重来，陷入焦虑。
 - 正确路径： 思考 -> 排序算法？ -> 分析复杂度 -> 设计 -> 编码
 - 思考： 我需要的是部分排序，完全排序做了太多无用功。
 - 搜索范式：有什么算法能高效解决“选择”问题？ -> 快速选择（QuickSelect）算法，平均 $O(n)$ 。
 - 设计伪代码，理清partition逻辑和递归边界。
 - 编码： 将清晰的伪代码转化为具体语言。

举例：线性时间选择

问题描述：给定线性序集中 n 个元素和一个整数 k ，要求找出这 n 个元素中第 k 小的元素。

- 特殊情况
- 期望为线性时间的选择算法
- 最坏情况为线性的选择算法

- 特殊情况： $k=1$ （最小值）或 $k=n$ （最大值）
- 算法：依次遍历每个元素，并记录当前最小/大元素

```
MINIMUM(A)
1  min = A[1]
2  for i = 2 to A.length
3      if min > A[i]
4          min = A[i]
5  return min
```

- 特殊情况：同时找最小值和最大值
- 算法：
 - 1) 分别独立地找出最小值和最大值，共需 $2n-2$ 次比较
 - 2) 成对处理输入元素：将一对输入元素相互比较，把较小的与当前最小值比较，把较大的与当前最大值比较。即每两个元素共需3次比较。共需 $3\lfloor n/2 \rfloor$

- 特殊情况：输入数据全是整数
- 线性时间排序算法

问题描述：给定线性序集中 n 个元素和一个整数 k ，要求找出这 n 个元素中第 k 小的元素。

一般情况的解法

解法1：暴力法

解法2：归并排序

解法3：快速排序

解法4：期望为线性时间的选择算法

解法5：最坏为线性时间的选择算法

误区二：缺乏抽象，无法形式化

- 现象：困在问题表面，无法看到计算本质。
- 解释：现实世界问题虽然纷繁复杂，但计算模型是有限的。抽象是将具体问题映射到已知计算模型（如图、树、集合）的关键一步。
- 案例：社交网络中，如何找到A和B之间的最短好友关系链？
 - 表象困惑：“我要查A的好友，再查好友的好友...怎么避免循环？怎么记录路径？好复杂！”
 - 抽象建模：（1）识别实体：用户 $\rightarrow$ 顶点；（2）识别关系：好友关系 $\rightarrow$ 边；（3）建立模型：整个网络 $\rightarrow$ 图；（4）定义问题：求两个顶点间的最短路径。
 - 解决方案：**BFS**（广度优先搜索）是解决无权图最短路径的标准工具。问题瞬间从“无从下手”变为“直接应用经典算法”。

例1、澳大利亚地图染色问题(1)

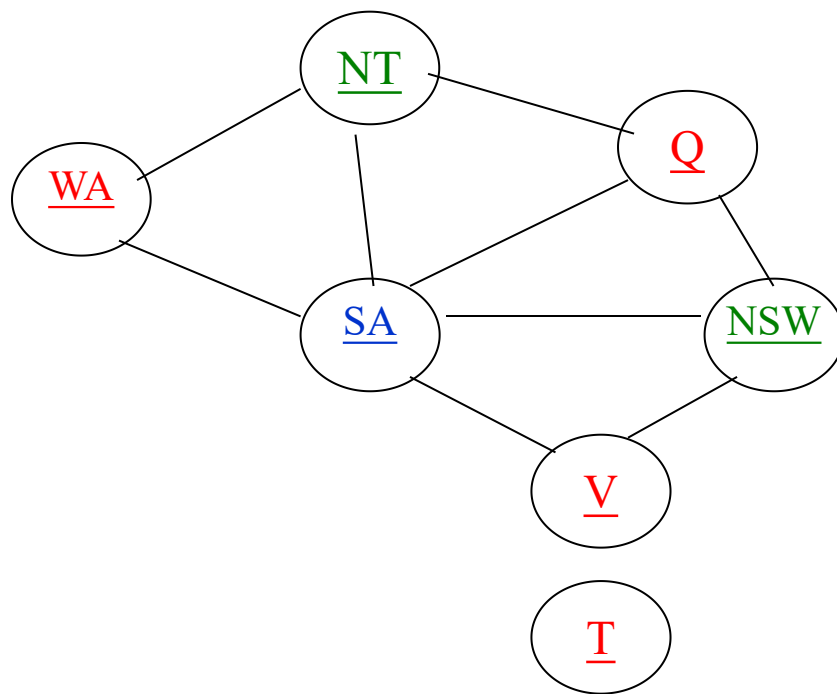
- 澳大利亚地图染色问题：用红绿蓝3色标出各省，相邻者颜色不同。



例1、澳大利亚地图染色问题(2)

- 对应于澳大利亚地图的约束图，相互关联的节点用边连接。

- 西澳大利亚 – WA
- 北领地 – NT
- 南澳大利亚 – SA
- 昆士兰 – Q
- 新南威尔士 – NSW
- 维多利亚 – V
- 塔斯马尼亚 – T



- 一组满足约束的完全赋值: { WA=R, NT=G, Q=R, SA=B, NSW=G, V=R, T=R }

例2：素数环问题

- 针对一个整数 n ，把从1到 n 的数字无重复的排列成环，且使每相邻两个数（包括首尾）的和都为素数，则称为素数环。
- 要求：
 - （1）请选择合适的数据结构对该问题进行建模，并给出建模的基本思路。
 - （2）可使用什么算法求解该问题？请给出求解该问题的基本思想。
 - （3）请给出算法的伪代码，并在重要步骤上给出注释。

例3：传教士和野人问题

- 例 传教士和野人问题

<http://www.973.com/p45383>

— 解：

— 状态表示

- (m, w, B)
- 初始状态： $(3, 3, 1)$
- 目标状态： $(0, 0, 0)$

Left Bank



例3：传教士和野人问题

– 操作符（产生式规则）

– 左岸往右岸运载产生式

- L_{10} if(M,W,1) then (M-1,W,0) (左往右运1传教士)
- L_{01} if(M,W,1) then (M,W-1,0) (左往右运1野人)
- L_{11} if(M,W,1) then (M-1,W-1,0) (左往右运1传教士和1野人)
- L_{20} if(M,W,1) then (M-2,W,0) (左往右运2传教士)
- L_{02} if(M,W,1) then (M,W-2,0) (左往右运2野人)

例3：传教士和野人问题

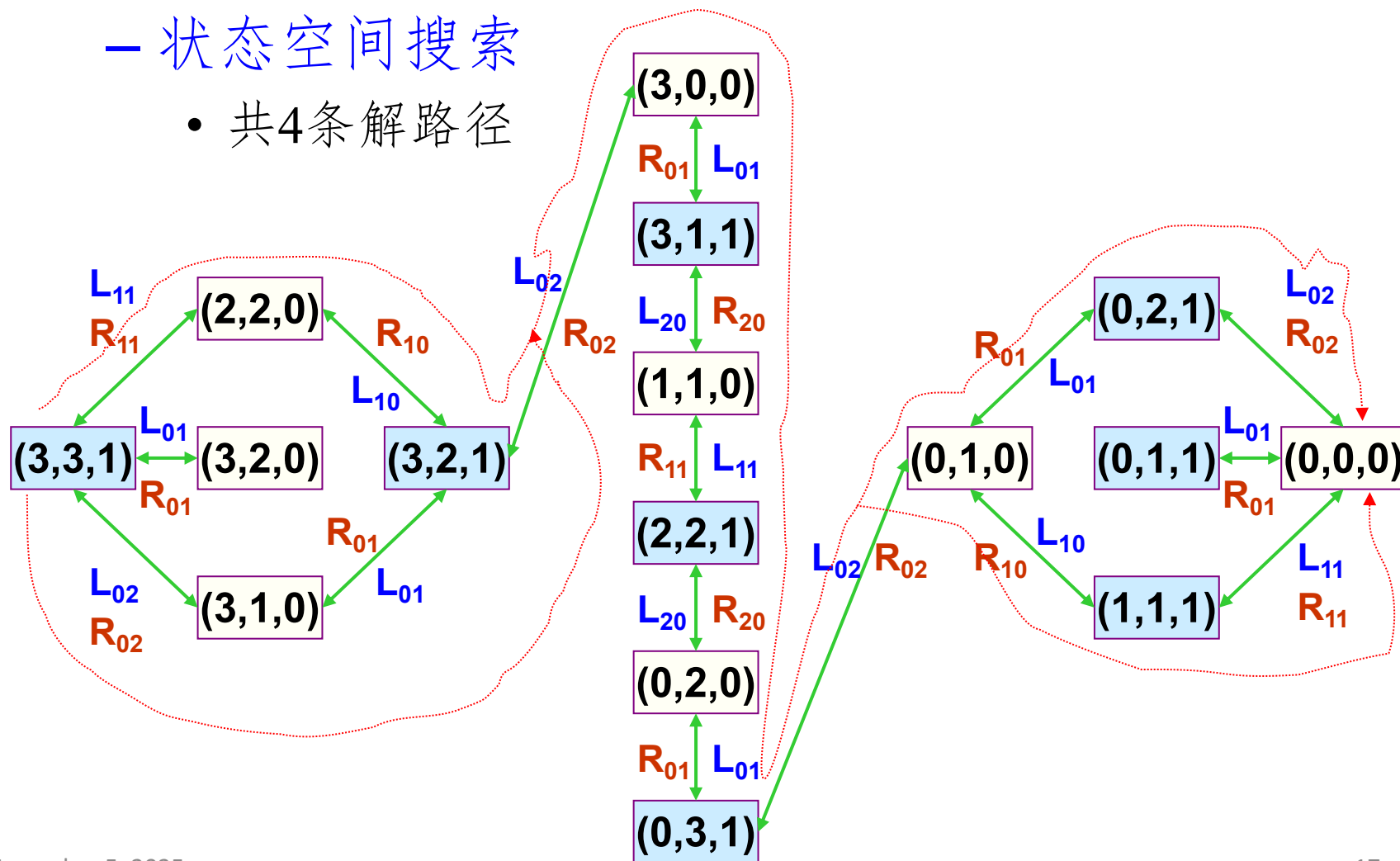
— 右岸往左岸运载产生式

- R_{10} $\text{if}(M,W,0) \text{ then } (M+1,W,1)$ (右往左运1传教士)
- R_{01} $\text{if}(M,W,0) \text{ then } (M,W+1,1)$ (右往左运1野人)
- R_{11} $\text{if}(M,W,0) \text{ then } (M+1,W+1,1)$ (右往左运1传教士和1野人)
- R_{20} $\text{if}(M,W,0) \text{ then } (M+2,W,1)$ (右往左运2传教士)
- R_{02} $\text{if}(M,W,0) \text{ then } (M,W+2,1)$ (右往左运2野人)

例3：传教士和野人问题

— 状态空间搜索

- 共4条解路径



例4：一般的传教士和野人问题

- 一般的传教士和野人问题 (**Missionaries and Cannibals**)：有 N 个传教士和 N 个野人来到河边准备渡河。河岸有一条船，每次至多可供 K 人乘渡。问传教士为了安全起见，应如何规划摆渡方案，使得任何时刻，在河的两岸以及船上的野人数目总是不超过传教士的数目，但允许在河的某一岸只有野人而没有传教士。
- 设 $N=5$ ， $K \leq 3$ 。在某一时刻，在河的左岸的传教士人数用 M 表示，野人数用 C 表示。 $B=1$ 表示船在左岸， $B=0$ 表示船在右岸。

误区三：不善于检索，重复造轮子



- 结论：遇到的大部分问题，早已有优秀的解决方案。
- 解释：算法领域的核心是复用和创新。不知道已有解决方案，不仅效率低下，而且容易写出充满漏洞的代码。精准的搜索能力是核心素养。
- 案例：给定两个序列 $x[1 \dots m]$ 和 $y[1 \dots n]$,找出他们两者一个最长公共子序列？
 - 自己造轮子: 对每个子序列，检查 = $O(n)$ 时间；序列 x 有 2^m 个子序列(每个长度为 m 的位向量对应一个 x 的子序列).最坏情况下的运行时间 = $O(n2^m)$
 - 发现：动态规划算法
 - 学习：理解最优子结构和公共子问题
 - 应用：站在了巨人的肩膀上

误区四：证明缺失，盲目自信

- 结论：“感觉可行”是算法设计中比较危险的一句话。
- 解释：算法的正确性绝非显而易见（如贪心法）。没有严格的数学证明，算法只是一个“猜想”，在特定输入下可能会崩溃。
- 案例：区间调度——如何安排尽可能多的不重叠会议？
 - 直觉方案：选择最早开始的会议？→ 反例：一个最早开始但开一整天的会议，会堵死所有后续机会
 - 正确方案：选择最早结束的会议
 - 为什么？因为这为后面留下了尽可能多的时间。但必须证明！
 - 证明思路：
 - 贪心选择性质：证明第一步选择最早结束的会议 o 是全局最优解的一部分。
 - 最优子结构：证明在选择了 o 之后，剩下的问题是在 o 结束之后才开始的时间段里，寻找最优解。这个子问题的最优解 + o = 原问题最优解。
 - 没有证明，你的算法就没有可信度。

建议1：伪代码——设计的蓝图

- 核心挑战：从算法思路到程序实现的鸿沟
 - 清晰的算法设计 $\neq$ 简洁正确的代码实现，中间存在一道“鸿沟”。
- 结论：伪代码是设计与实现之间的完美缓冲。
- 解释：伪代码用自然语言和代码结构的混合体来描述算法逻辑，忽略语法细节，聚焦于流程、条件和循环等高层次结构。是与人、与己沟通设计思想的最佳工具。
- 例子：快速排序的伪代码 vs C++代码
 - 快速排序的核心思想？
 - 举例：数组：537416，升序排序
 - (1) 给出使用快速排序的执行过程
 - (2) 针对 (1) 的步骤，写出对应的伪代码？
 - (3) 思考题：伪代码是否正确怎么验证？
 - (4) 思考题：是否可以改进？

建议1：伪代码——设计的蓝图

- 结论: 伪代码是设计与实现之间的完美缓冲。
- 解释: 伪代码用自然语言和代码结构的混合体来描述算法逻辑, 忽略语法细节, 聚焦于流程、条件和循环等高层次结构。是与人、与己沟通设计思想的最佳工具。
- 例子: 快速排序的伪代码 vs C++代码

```
int partition(int arr[], int low, int high) { ... } // 需要实现swap、指针移动等  
细节
```

```
void quicksort(int arr[], int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
        quicksort(arr, low, pi - 1);  
        quicksort(arr, pi + 1, high);  
    }  
}
```

建议1：伪代码——设计的蓝图

- 实用的建议：
 - 测试驱动开发思维：在写逻辑代码前，先思考并写下测试用例。
 - 例子：实现“二分查找”前，先假设一个小样本的数据，并在该样例中分析不同的测试情况：空数组、单元素数组、找不到目标、目标在开头/中间/末尾、有重复元素等。
 - 用测试用例定义了“边界”，让模糊的设计变得具体可测。
 - 模块化与接口化：将伪代码中的关键步骤转化为独立的函数/方法。
 - 实现快速排序时，先定义好`partition(arr, low, high) -> pivot_index`的函数声明，再去实现。
 - 好处：分解复杂度，易于调试和验证。
 - 增量实现与验证：不要一次性实现所有代码。实现一个模块，就立刻用准备好的测试用例验证它。
 - 例子：实现图算法时，先单独实现和测试“图的构建”模块，确保数据加载正确，再实现后面的BFS/DFS逻辑。
 - 好处：将问题局部化，能快速定位BUG的来源。
 - 防御性编程：在代码中主动检查假设是否成立。
 - 例子：在函数入口检查指针是否为空、索引是否越界。“如果这个输入不满足假设，我该怎么办？”（快速失败，返回错误）
 - 好处：避免隐藏的BUG，使程序更健壮。

快速排序一个有 n 个项的数组:

1. **分解:** 围绕**基准** x 将数组划分成两个子数组, 使得左边的子数组中的项 $\leq$, 右边子数组中的项 $\geq x$.



2. **解决:** 递归的对两个子数组排序.
3. **合并:** 很简单.

关键: 线性时间的划分函数.

快速排序的伪代码

QUICKSORT(A, p, r)

if $p < r$

then

$q \leftarrow \text{PARTITION}(A, p, r)$

QUICKSORT($A, p, q-1$)

QUICKSORT($A, q+1, r$)

直接调用: QUICKSORT($A, 1, n$)

划分函数

PARTITION(A, p, q) $\triangleright A[p \dots q]$

$x \leftarrow A[p]$ $\triangleright$ pivot = $A[p]$
 $i \leftarrow p$

for $j \leftarrow p + 1$ **to** q

do if $A[j] \leq x$

then $i \leftarrow i + 1$
 $\text{exchange } A[i] \leftrightarrow A[j]$

$\text{exchange } A[p] \leftrightarrow A[i]$

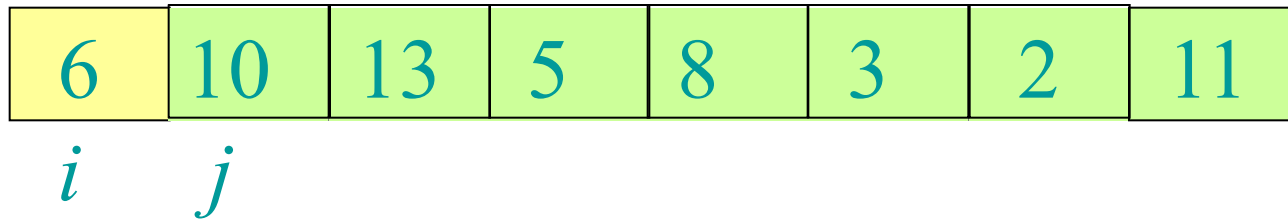
return i

n 个项的运行
时间= $O(n)$

不变量:



划分举例



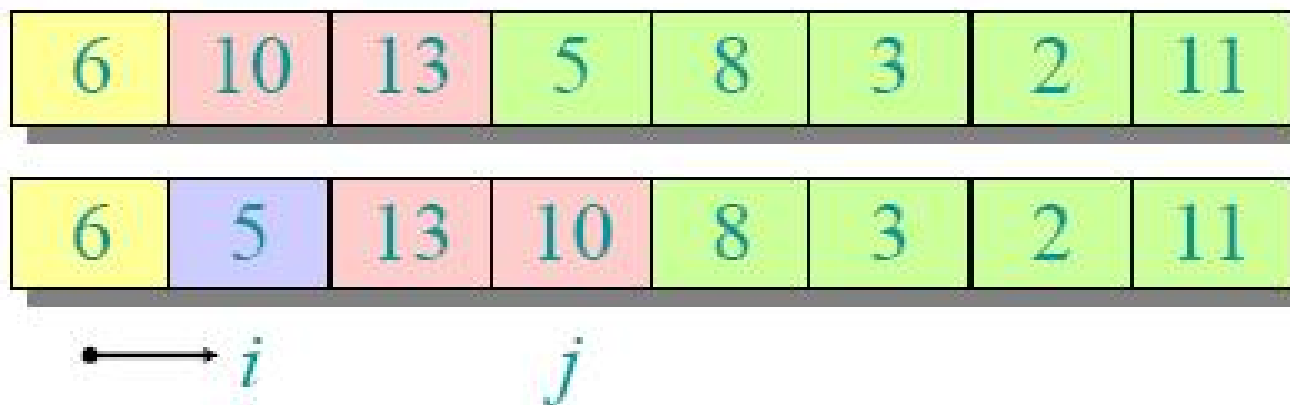
划分举例



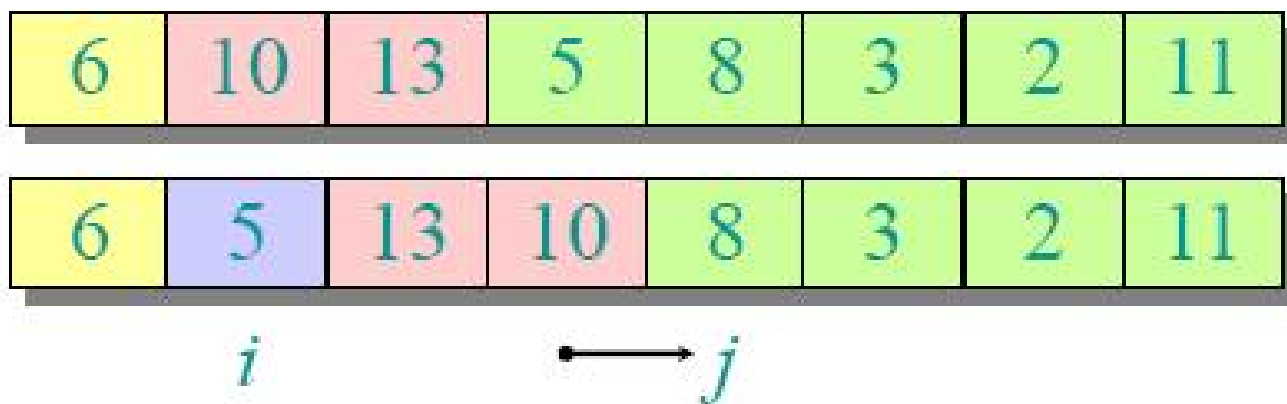
划分举例



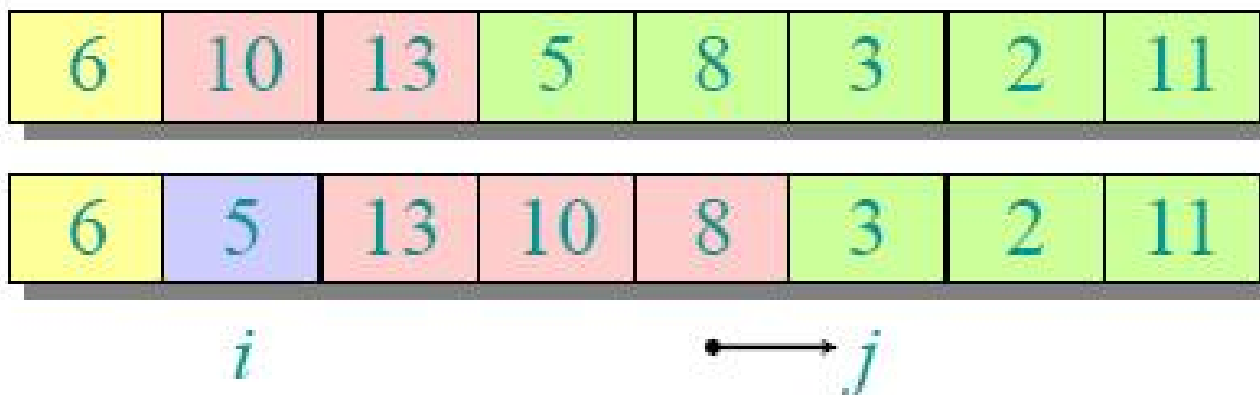
划分举例



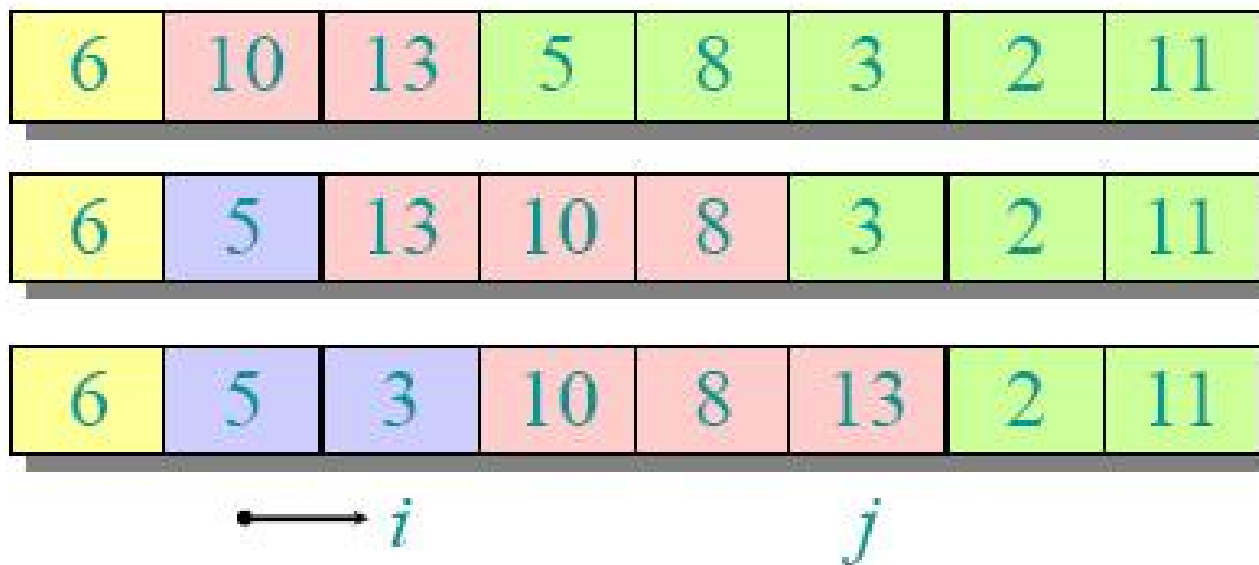
划分举例



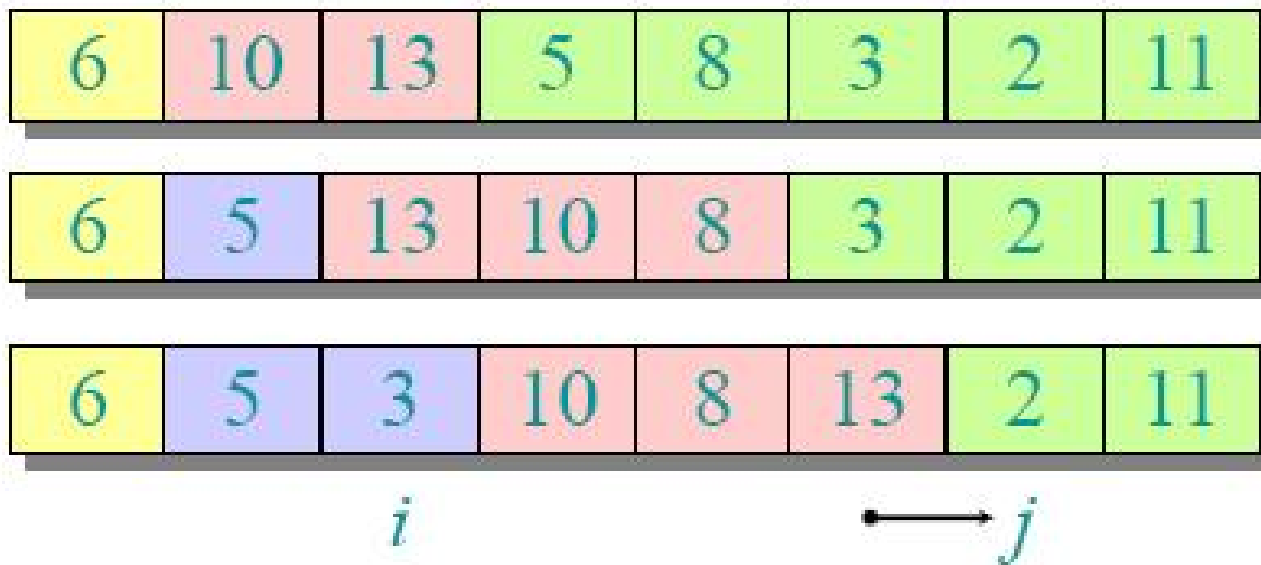
划分举例



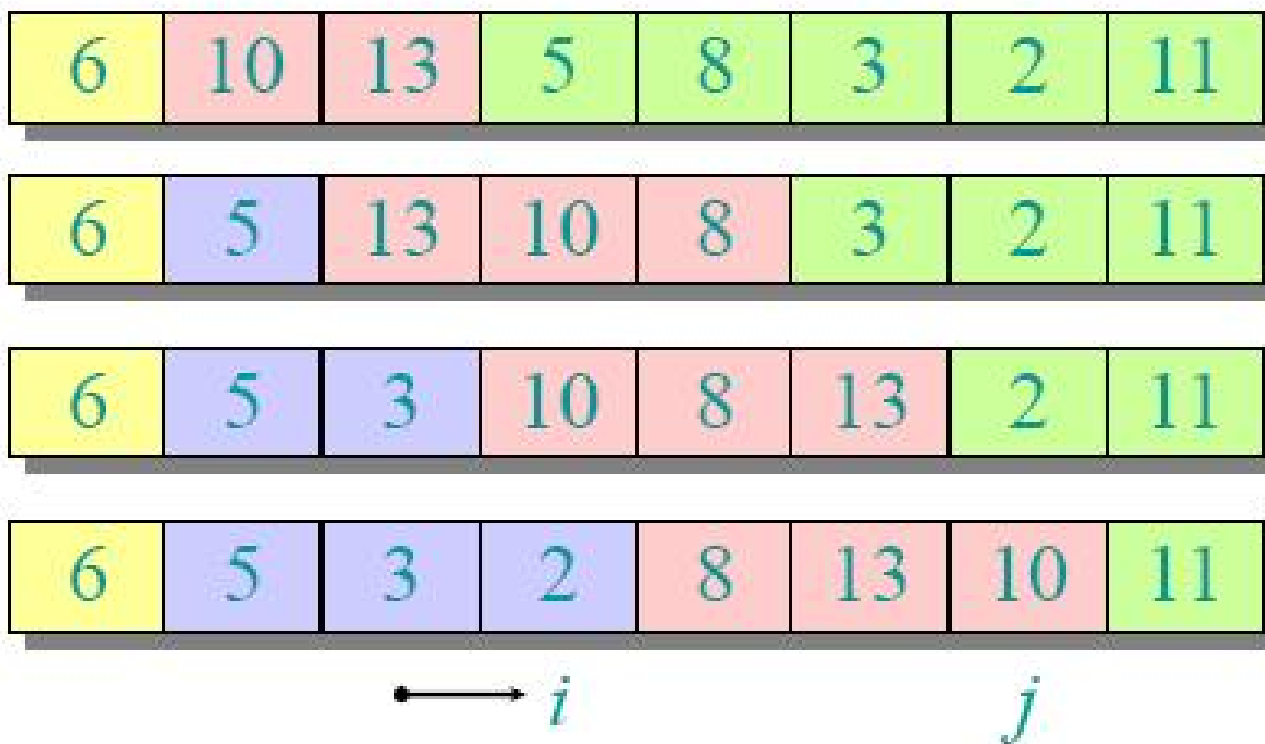
划分举例



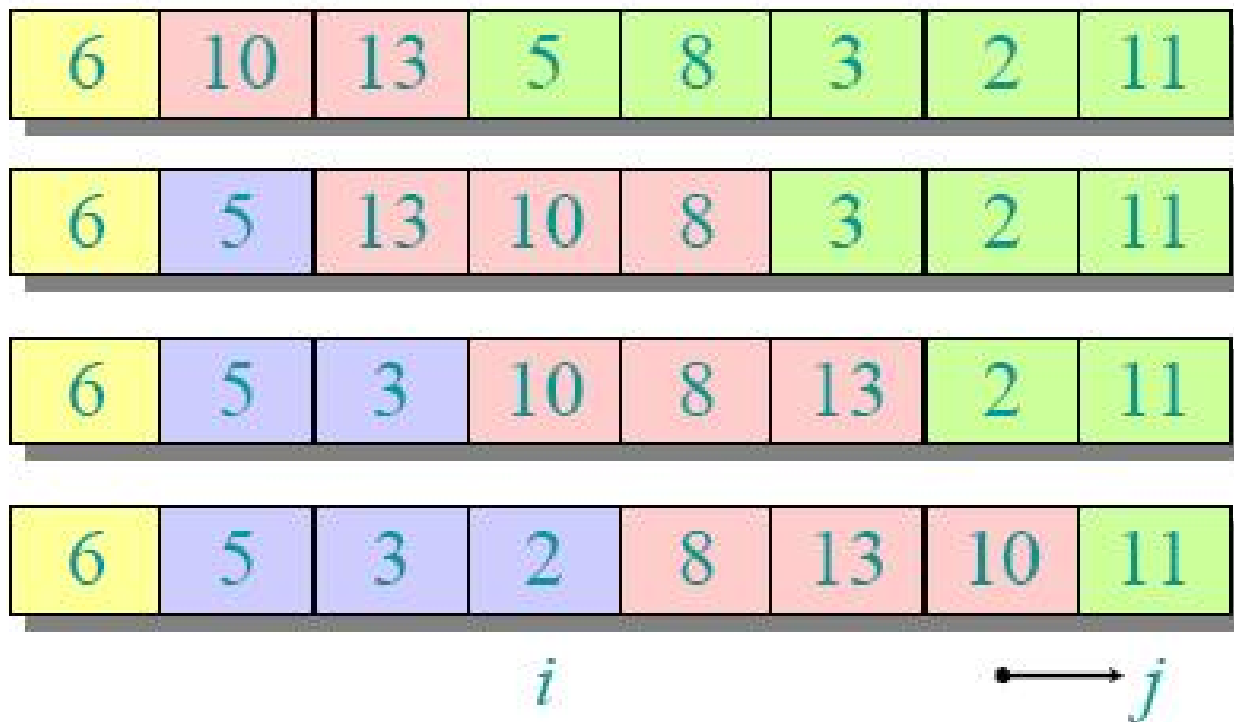
划分举例



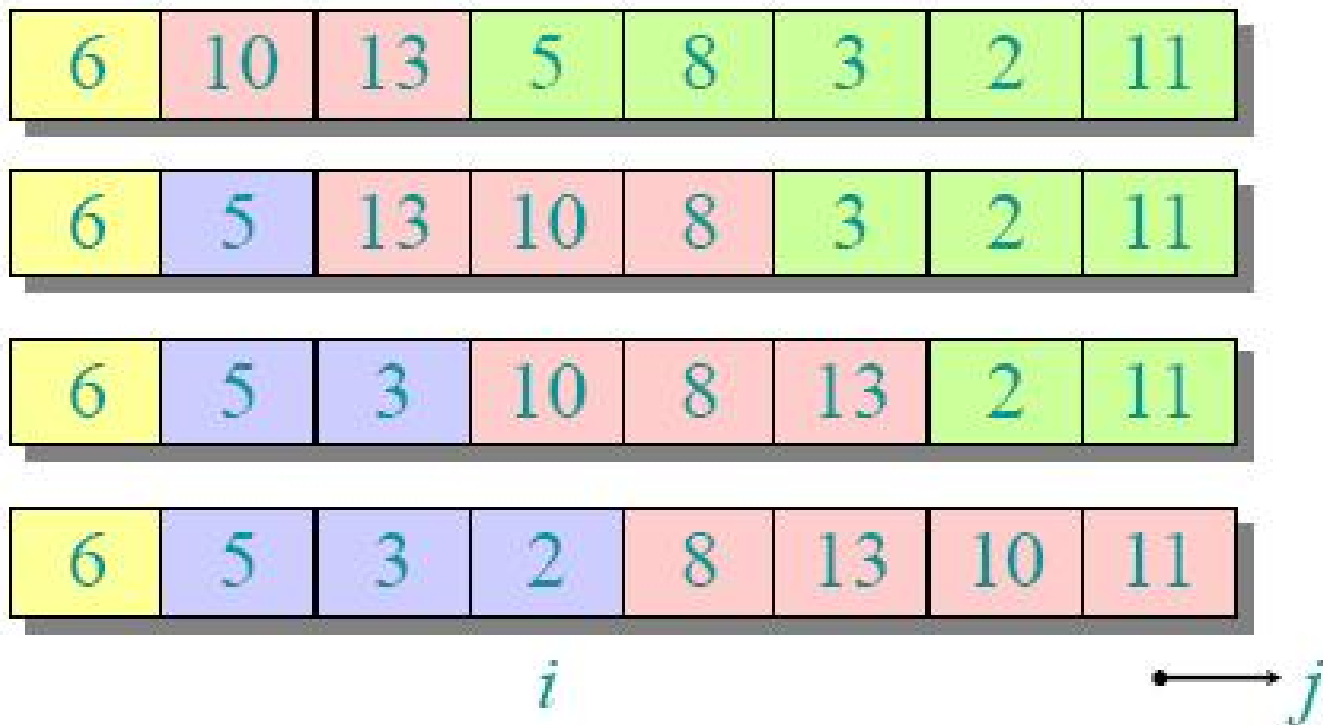
划分举例



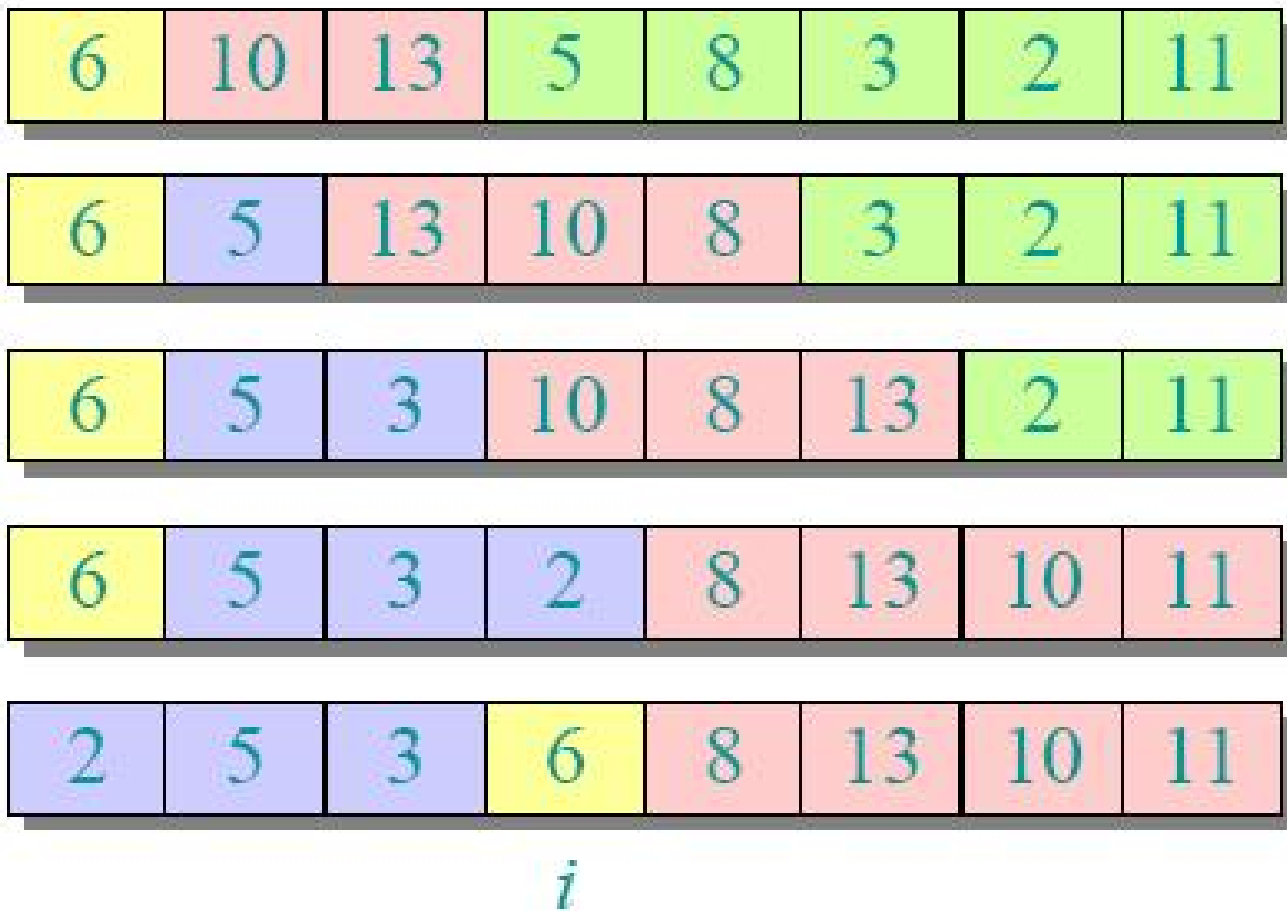
划分举例



划分举例



划分举例



建议2：建模与抽象

- 如何训练建模与抽象的能力？
 - “模式识别”：本课程会系统讲解各种经典算法范式（如分治、贪心、动态规划、回溯等）。
 - 大量案例训练：每讲一个范式，都会用多个问题来演示如何抽象和建模。

建议3：分析与证明

- 我们将学习：

时间复杂度：大O等时间复杂度记号、最坏/平均/最好情况分析。

正确性证明：循环不变式、数学归纳法、反证法等。

求解递归式、最优子结构、贪心选择性质等。

归并排序的过程（分治法）：

1. 分解：显而易见。
2. 解决：递归地对两个子数组排序。
3. 组合：线性时间合并。

$$T(n) = 2T(n/2) + \Theta(n)$$

子问题数目 子问题规模 *work dividing and combining*

建议4：资源搜索与利用

● 我们将实践：

- 如何将一个问题转化为搜索关键词。（例如，从“最短好友链”到“无权图最短路径 BFS”）
- 介绍关键的学习资源

MIT公开课资源：

<https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-046j-introduction-to-algorithms-sma-5503-fall-2005/>

bilibili资源（有字幕）：

<https://www.bilibili.com/video/av58608492/>

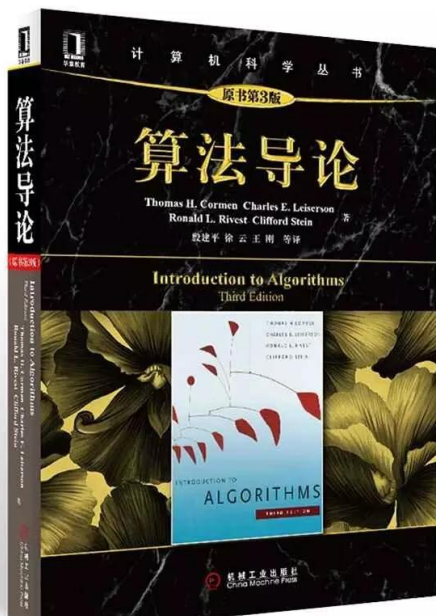
代码资源：

<https://github.com/>

<https://huggingface.co/papers/trending>

在线工具书：

<https://www.huaxiaozhuan.com/>



- 算法与计算思维



技能：会做

能力：做得好

思维：认知、方法论

- 2006年3月周以真(Jeannette M. Wing, 卡内基·梅隆大学计算机系系主任)首次提出Computational Thinking的概念：

运用计算机科学的基础概念去求解问题、设计系统和理解人类的行为，它包括了涵盖计算机科学之广度的一系列思维活动。

2010年，周以真教授又指出计算思维是与形式化问题及其解决方案相关的思维过程，其解决问题的表示形式应该能有效地被信息处理代理执行。

- 数学思维与工程思维的互补与融合：抽象与实现

	实验思维	理论思维	计算思维
起源	物理学	数学	计算机科学
过程步骤	1.实验观察归纳建立简单数学公式 2.导出数量关系 3.实验验证	1.定义概念 2.提出定理 3.给出证明	1.建模(约简、嵌入、转化、仿真、...) 2.抽象与分解,控制系统复杂性 3.自动化实现...
特点	解释以往现象 无矛盾 预见新的现象	公理集 可靠协调推演规则 正确性依赖于公理	结论表示有限性 语义确定性 实现机械性

- 计算思维吸取了问题解决所采用的一般数学思维方法，现实世界中巨大复杂系统的设计与评估的一般工程思维方法，以及复杂性、智能、心理、人类行为的理解等的一般科学思维方法。
- 计算思维建立在计算过程的能力和限制之上，由人由机器执行。计算方法和模型使我们敢于去处理那些原本无法由个人独立完成的问题求解和系统设计。

- 算法与计算思维：
 - 算法课程是训练计算思维的重要课程；涉及到对问题的抽象，建模，设计好的求解方法，复杂性的控制，...
 - 可计算性与计算复杂性：形式化、确定性、有限性，抽象与逻辑证明
 - 算法设计与分析：抽象建模、归约、正确性证明、效率分析、...

- 算法基础知识
- 算法设计与分析技术
 - 分治策略
 - 动态规划
 - 贪心法
 - 回溯法
 - 分支限界法
- 高级专题

本门课程的学习目标

- 掌握算法设计的基本策略，并能针对实际问题合理运用算法设计策略来设计算法求解。
- 理解算法设计思路，建立合理的计算思维。

- 教学QQ群：794861956



群名称:2025学年本科算法
群 号:794861956

总结：本课程的学习目标与期望

1. 从【Coder】升级为【Designer】：养成先设计（伪代码）、后实现的习惯。
2. 获得一双【X-Ray眼】：能穿透问题描述，看到背后的数据结构和算法模型。
3. 建立一个【豪华工具箱】：对大多数常见问题，能快速反应出可能适用的算法范式 and 具体算法。
4. 掌握一项【核心论证能力】：能为自己的设计提供严谨的复杂度分析和正确性论证。
5. 学会【高效学习】：掌握持续自学和探索新算法的方法。



合肥工业大学
HEFEI UNIVERSITY OF TECHNOLOGY

Q/A?



合肥工业大学数据挖掘与智能计算“千人计划”研究团队
HFUT Data Mining and Intelligent Computing Laboratory